Mainrun LLM Report

by Arsh Chawla

Abstract. We train a 46M-parameter language model on Hacker News titles under a fixed data and compute budget, cutting validation loss from a 1.7803 baseline to 1.1416. Phase A builds a strong backbone through iso-parameter, iso-token coordinate descent, tuning one component at a time—optimizer, schedule, tokenizer, shape, masking, batching, and regularisation—so every comparison stays fair at equal capacity. Phase B then probes more ambitious ideas: alternative attention mechanics (output-gating, value-residual, differential), cross-layer residual paths, and a diagnosis of the train–val gap that pinpoints overfitting as the binding constraint. The decisive win is R-Drop consistency regularisation, which reopens a capacity×regularisation frontier, paired with decoupled weight decay on the Muon-optimised matrices. We build a BERT-style MLM critic to score generations by pseudo-log-likelihood to measure autoreggresive decoding capabiltities (as a sanity check). We report the many ideas that failed, including multi-token-prediction, NorMuon, and many others. We close by building a dependency-light CPU inference engine in Rust around the final architecture.

Contents

1 Introduction

We optimise a small language model for Hacker News titles on a fixed dataset and compute budget. The repository is built around a single benchmark engine `bench.py` and a declarative experiment configuration: every experiment is a `Sweep` that varies a few axes while holding everything else fixed, so each design decision is tested in isolation. Comparisons are kept fair by construction—sweeps can be run *iso-parameter* (a solver picks `d_model`/`n_head` to hit a fixed parameter budget at each point, so capacity is held constant even as the tokenizer or shape changes) and *iso-tokens/step* (batch size is derived from a fixed `tokens_per_step`). Every run is extensively logged, with the `model.pt` saved for further analysis. The engine, repository layout and hardware are described in full in Appendix A.

We optimise a single scored metric: teacher-forced next-token validation loss, a per-character cross-entropy (nats/char) on a held-out split that stays comparable across tokenizers. This never tests the key functionality of

the model, autoreggresive generation, therefore we devise two *secondary* checks (purely for sanity)—qualitative analysis of generated titles, and a learned BERT-style critic that scores them by pseudo-log-likelihood—to confirm that lower loss is moving toward coherent, hn title-like text. The full evaluation protocol is given in Appendix B.

Our approach proceeds in two phases. In **Phase A** we establish a strong baseline via coordinate descent across the key hyperparameters and grounded architectural changes.. In **Phase B** we then ablate that best model against increasingly custom and novel ideas in the search for performance and efficiency.

2 Coordinate Descent (Phase A)

The goal of phase A is to identify a strong starting point for the HN LLM. This involves design decisions like the tokenizer, model size/shape, context window etc. This is our optimal prior for implementing more interesting ideas, mitigating the need to sweep all hyperparameters upon every change.

We optimise by component: sweep one component, lock its optimum, and carry it forward. This is coordinate descent [1], and it is far cheaper than a joint search, whose cost grows multiplicatively in the number of hyperparameters. It operates under the assumption that the components are roughly separable; where two interact (tokenizer and vocab_size, or block and batch) we do joint sweeps.

We run the axes from the most global to the most specific. The optimizer and schedule come first, since they govern how well any model trains; fixing them means every later comparison is made on a properly trained model, rather than one confounded by an under-tuned optimizer. We then lock the architecture (pulling tried ideas from the Llama series). The tokenizer is next, as it defines the units the model predicts, i.e. the language itself, and hence the distribution that capacity is later spent on. With the language fixed we search shape (how to allocate parameters across depth and width), then the data-handling choices (masking, block size, batching), and finally regularisation, whose optimum depends on the capacity chosen above.

The baseline model (0_baseline) achieved $\mathcal{L} = 1.7803$ with poor generations (see Appendix D).

Prior to coordinate descent we fix the attention implementation, replacing the baseline's attention with GPU-based `FlashAttention`. This only changes memory and speed, not loss. Doing this first makes every Phase A run as fast as the hardware allows, making search more affordable.

2.1 Optimizer

The optimizer governs how efficiently the model trains under a fixed epoch budget. We split the search in two: the gradient-descent algorithm and its learning rate (1_optim), then the learning-rate schedule (2_wsd_decay).

2.1.1 Algorithm and learning rate

Motivation. We compare three optimizers, each across a range of learning rates. **SGD** with momentum is the simplest baseline but tends to underfit transformers. **AdamW** [2] adds per-parameter adaptive moments with decoupled weight decay and is the standard choice for language models. **Muon** [3] is a recent optimizer that orthogonalises each 2D weight matrix's momentum update, via a Newton-Schulz iteration. Muon applies only to 2D matrices, so we run it as a hybrid: Muon on the transformer weight matrices, AdamW on the embeddings, tied head, and norm/bias parameters.

Results. The figure below plots validation loss against learning rate, one curve per optimizer. SGD underfits badly at every rate (best 1.68). AdamW and the Muon-hybrid are both strong, but Muon wins convincingly at lr = 0.01 (val 1.2773, against AdamW's best of 1.3446). We analyse autoreggresive generation as a sanity check, which is consistent with the loss, avalaible at Appendix D.

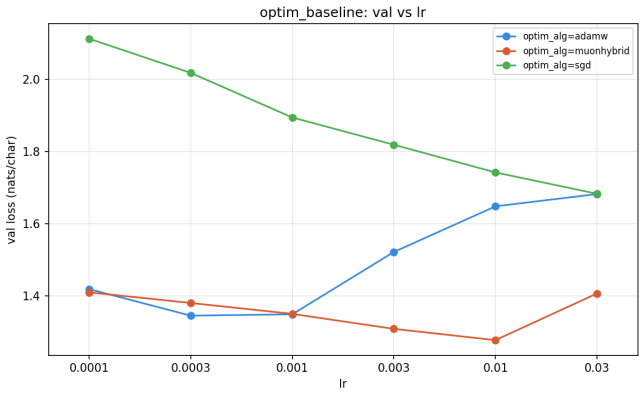

Figure 1: Optimizer sweep: validation loss vs learning rate

Choice. Muon-hybrid at lr = 0.01: the clear best on validation, and the most coherent generations of the three.

2.1.2 Schedule (WSD)

Motivation. Cosine annealing is the default but commits to a fixed decay from the first step. Warmup-Stable-Decay (WSD) [4] instead holds the peak learning rate on a long stable plateau and anneals only over a short tail, keeping most of training at high LR and decoupling the decay length from the run length. We sweep the decay fraction and the decay shape (linear, cosine, sqrt).

Note. With only 7 epochs there is little room for the schedule to matter; we expect a small effect.

Results. The figure below plots validation loss against decay fraction, one curve per decay shape. As expected the spread is small: every WSD setting lands between 1.244 and 1.265, all below the cosine baseline of 1.2773. A short sqrt decay over the final 10% is nominally best (1.2438).

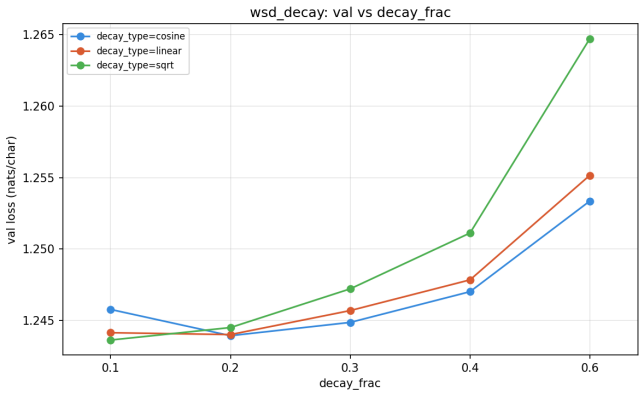

Figure 2: WSD sweep: validation loss vs decay fraction

Choice. WSD with a sqrt decay over the final 10% (val 1.2438). Unlike the algorithm sweep, this run has no internal baseline, so we anchor against the cosine result from the optimizer run (1.2773): every WSD setting beats it, so adopting WSD is a clear win (1.2773 to 1.2438). The choice of decay shape and fraction barely matters (within noise), consistent with the few-epoch caveat above.

2.2 Architecture (Llama)

With the optimizer fixed, we optimise key architectural blocks. Four of the changes below are standard in the Llama family [5] (RMSNorm, RoPE, SwiGLU, removing biases); we also test QK-normalisation, a stabilisation trick used in several Llama-lineage models.

- **RMSNorm** [6] replaces LayerNorm, normalising by root-mean-square only and dropping the mean subtraction and the bias. It is a cheaper, simpler normalizer with no learned shift.

```
# RMSNorm https://arxiv.org/abs/1910.07467
class RMSNorm(nn.Module):
    def __init__(self, dim, eps=1e-6):
        super().__init__()
        self.eps = eps
        self.weight = nn.Parameter(torch.ones(dim))
    def forward(self, x):
        x = x * torch.rsqrt(x.pow(2).mean(-1, keepdim=True
    ) + self.eps)
        return x * self.weight
```

- **RoPE** [7] replaces learned absolute position embeddings by rotating queries and keys by an angle proportional to position. The rotation makes the `q·k` dot product depend only on the relative offset `(i - j)`, meaning position is encoded relatively and integrates directly into attention. We use the frequency base of 10000.
- **SwiGLU** [8] replaces the GELU MLP with a gated unit. It uses three projections instead of two, with the hidden width set to `8/3·d` to keep the parameter count matched:

$$\mathrm{SwiGLU}(x) = \big(\mathrm{SiLU}(xW_1) \odot xW_3\big)\, W_2$$

- **Bias** removal: Llama drops biases from linear layers and norms. We test a fully bias-free block.
- **QK-norm** [9] applies RMSNorm to queries and keys before attention, bounding the attention logits for stability.

Method. We adopt a change if it lowers loss, but loss alone is one-dimensional and several deltas here are within noise. We therefore also weigh theoretical grounding, and prioritise changes which simplify the model.

Change (vs best so far)	Val loss	Δ	Decision
Baseline (post-WSD)	1.2438	ref	—
RoPE	1.2255	−0.0183	adopt
RMSNorm	1.2435	−0.0003	adopt
Bias off	1.2439	+0.0001	reject
SwiGLU	1.2446	+0.0008	reject
QK-norm	1.2472	+0.0034	reject

Choice. We adopt **RoPE** and **RMSNorm**. RoPE is a clear win on loss (1.2438 to 1.2255). RMSNorm is flat on loss but is a well-grounded simplification of LayerNorm (no mean, no bias) and does not hurt, so we keep it. The other three add complexity without a payoff: SwiGLU (three projections rather than two) is slightly worse, and removing biases or adding QK-norm give no improvement, so we drop all three.

2.3 Tokenizer

The tokenizer is a critical component as it defines the units the model predicts. We sweep a range of tokenizer algorithms vocabulary sizes.

Iso-parameter setup. Vocabulary size and tokenizer type change the parameter count directly, mostly through the embedding/unembedding ($V \cdot d$ parameters). Scaling literature counts these toward total size (they affect capacity, despite being a lookup [10]), so to compare tokenizers fairly we hold the *total* parameter count fixed at $N = 32$M and solve for the width that hits it. With L layers, width d, and vocab V, total parameters are

$$N = \underbrace{12\,L\,d^2}_{\text{transformer}} + \underbrace{V\,d}_{\text{embedding}},$$

which we invert for d at each (V, L):

$$d = \frac{-V + \sqrt{V^2 + 48\,L\,N}}{24\,L}, \qquad n_{\text{head}} = \mathrm{round}(d/64).$$

We fix $L = 6$ (from initial baseline) and vary only the width. Scaling studies find total size dominates shape [11], so holding depth costs little and saves a dimension of search. This iso-parameter approach is the fairest comparison available given we cannot vary the data or compute (epochs and dataset are fixed).

Tokenizers. All four share a byte-level pre-tokenizer (a common byte alphabet, so the comparison is fair):

- **BPE** [12] builds the vocabulary bottom-up, greedily merging the most frequent adjacent pair.

- **WordPiece** [13] merges bottom-up as well, but picks the merge that most increases the training-corpus likelihood rather than raw frequency.
- **Unigram** [14] works top-down: it starts from a large candidate set and prunes tokens to maximise corpus likelihood under a unigram language model, keeping the most useful subwords.
- **SuperBPE** [15] is two-stage BPE that, after learning subwords, keeps merging *across* whitespace into multi-word "superword" tokens, raising bytes-per-token compression. We suspect **SuperBPE** will be strong given the prevalence of terms such as *open-source* which may benefit as a singular token. HF tokenizers do not support cross-whitespace merges, so this required us to patch a Rust 'superbpe' fork of `tokenizers` (under `tools/`).

Results. The figure below plots validation loss against vocabulary size, one curve per tokenizer. SuperBPE is surprisingly poor and only gets worse as the vocabulary grows (1.285 at 8k up to 1.435 at 96k): its superword tokens, built to compress long text, end up having an overall negative impact on the decoder. BPE and WordPiece are comparable, both strongest near 64k (1.223 and 1.217). Unigram is the clear winner, lowest at a 16k vocabulary with $\mathcal{L} = 1.2005$.

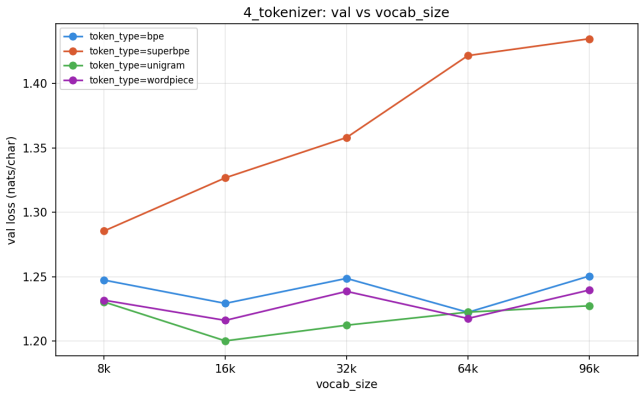

Figure 3: Tokenizer sweep: validation loss vs vocabulary size

Choice. Unigram at a 16k vocabulary ($\mathcal{L} = 1.2005$): the best of the four, and a small vocabulary keeps embedding parameters modest, leaving more of the fixed budget for the transformer.

2.4 Model Shape/Size

With the tokenizer fixed, we search the parameter shape: we vary depth (n_{layer}) against width (d_{model}), and read validation loss against N with one curve per depth (5_shape). Noteably we make the assumption that scaling the model size and shape will not invalidate previous findings (running assumption of coordinate descent).

Motivation. Width and depth add capacity differently. Width is the size of each token's representation, so a wider model can encode more features per token, i.e. more meaning. Depth is the number of attention-and-MLP rounds, so a deeper model attends and composes over more steps, building higher-order relationships between tokens. At fixed parameters the two trade off; scaling work finds the optimum is shape-insensitive to first order [11], with a mild, task-dependent preference [16].

Results. The figure below shows one U-shaped curve per depth. Every curve bottoms out on the narrow side (around $d_{\text{model}} = 384$, i.e. 16 to 40M params), well short of the widest settings, which only raise loss: with data and epochs fixed, wide models overfit. The deeper curves push this minimum narrower still and reach lower loss, so depth matters more than width here and narrow wins.

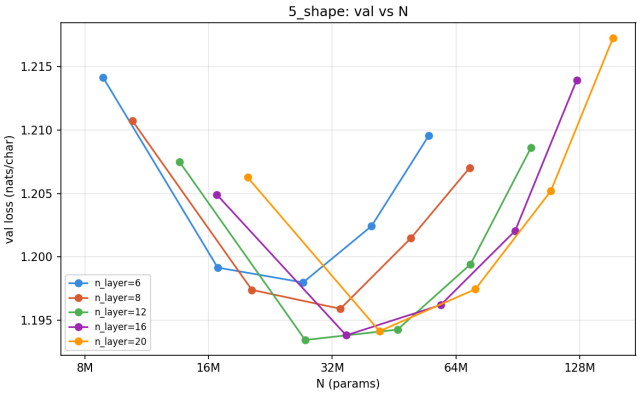

Figure 4: Shape sweep: validation loss vs N, one curve per depth

Size is not free: at a fixed token budget (7 epochs) wall-clock grows roughly linearly with parameter count, from ∼72 s at 9M to ∼916 s at 154M.

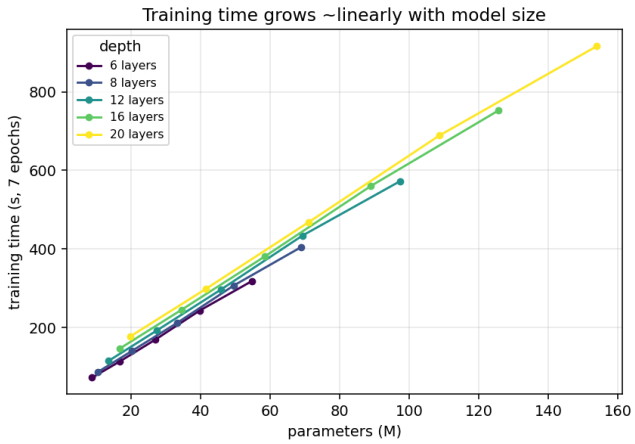

Figure 5: Shape sweep: training time vs parameter count, one curve per depth

Choice. $n_{\text{layer}} = 12$, $d_{\text{model}} = 384$ ($n_{\text{head}} = 6$), $N = 27.4M$ $\mathcal{L} = 1.1933$. This is the strongest performing model, and sits at the lower end of compute requirement.

2.5 Masking

Motivation. Training packs the corpus into one stream, `<title_1><eos><title_2><eos>...`, and slices it into `block_size` windows. A window therefore holds several

unrelated titles, and under a standard causal mask a token in one title attends back across <eos> into the titles before it. Two problems follow. First, that cross-title context is noise: titles are independent, so a previous headline says nothing about the current one. Second, and worse, the corpus is shuffled only once, so the same titles sit next to each other every epoch and the model can memorise these fixed cross-title co-occurrences. At inference each title is generated on its own (primed from <eos>, with no real prior-title context), so any cross-title cue learned in training is spurious and hurts generation.

To see this directly, we visualise one layer's head-averaged attention on a packed block of titles (boundaries in green; the window holds three full titles and the start of a fourth). Attention is mostly local, but a clear share crosses the boundaries onto previous titles. This is unrelated context it has learned, which it will never have at generation time.

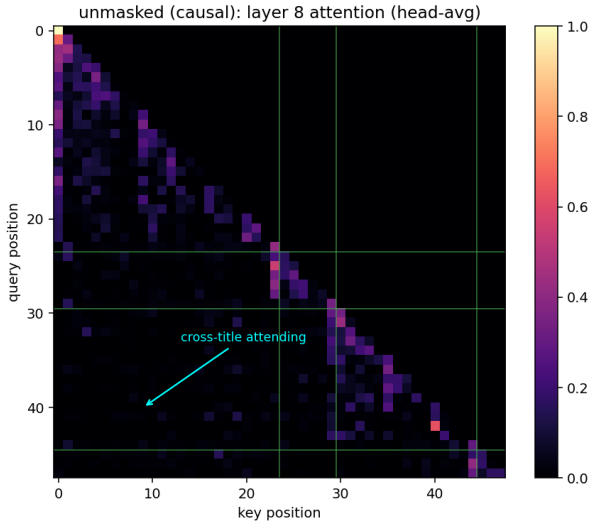

Figure 6: Unmasked attention: mass crosses the title boundaries

The fix. We want each title modelled in isolation, exactly as it is used at inference. This can be done several ways (resetting the stream per title, or padding each title to its own window), but those waste compute or alter the data. Instead we mask the attention: a token may attend only to earlier tokens within its own <eos>-delimited title, never across the boundary.

Each <eos> plays two roles: it is the end-of-title token the model learns to emit, and as the next input it opens the following title's block (its start token), which is why generation is primed from <eos>. We implement the mask with FlexAttention [17], which compiles an arbitrary masking rule into a fused, FlashAttention-speed kernel. The document id is recovered straight from the tokens as `doc_id = cumsum(token == <eos>)`, and attention is allowed where query and key share a `doc_id` and the key is not in the future.

Because the mask is rebuilt inside the attention module from the input tokens, it applies identically in training, validation, and generation. We never touch `evaluate()`: the same forward pass produces the same masking everywhere, so the change stays consistent and within the rules.

Results. Masking lowers validation loss, and more importantly removes a path the model should not be using. The same visualisation under masking is clean: attention is block-diagonal, confined to each title, with the per-title <eos> start token taking the role the sink had.

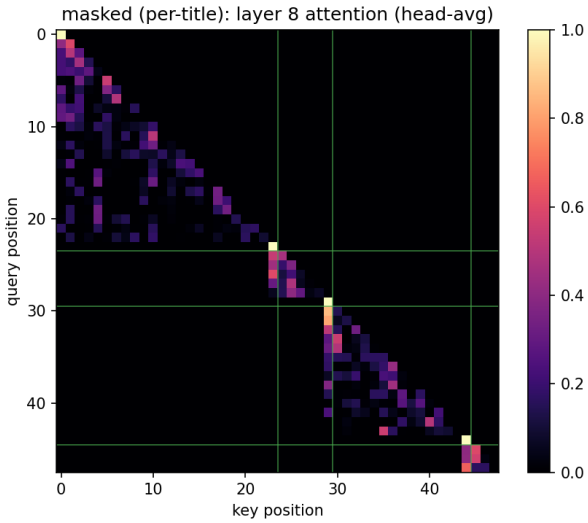

Figure 7: Masked attention: confined to each title

Title masking	Val loss
Off (causal)	1.1935
On (block-diagonal)	1.1924

Choice. Title masking on ($\mathcal{L} = 1.1924$). The loss gain is small at this block size, but the change is principled: it trains the model the way it is used and removes cross-title leakage. Its interaction with block size is the subject of the next two sections.

2.6 Batching (tokens per step)

Three quantities are linked by tps = batch × block, where tps is tokens per optimizer step. Over the fixed 7 epochs, tps sets the number of optimizer steps (more tokens per step means fewer, lower-noise steps), while block sets the context length. To separate the two we first fix the block and sweep the batch, which moves tps alone, to find the best tokens per step; then we fix that tps and sweep the block, which moves the batch inversely.

Tokens per step. Fixing block = 128 and sweeping batch (7_tokens_per_step) gives a clean optimum at tps = 4096 (batch 32): fewer tokens per step adds gradient noise, more spends the epoch budget on too few steps.

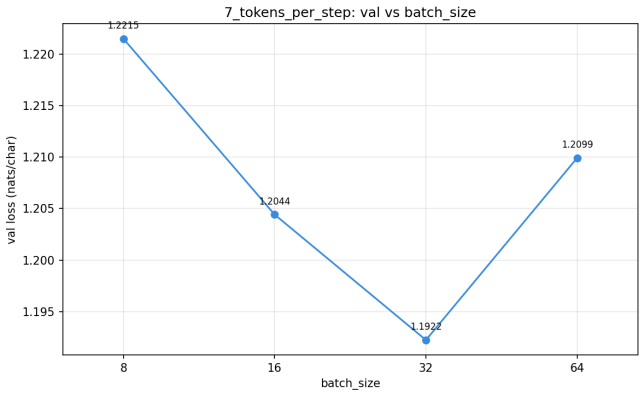

Figure 8: tps sweep: validation loss vs batch (block = 128, so tps = 128 x batch)

Block size. Fixing tps = 4096 and sweeping the block (8_block; batch adjusts to match), loss falls steadily as the block grows, since a larger window splits fewer titles at its boundaries. 512 is best ($\mathcal{L} = 1.1873$), but the gain over 256 ($\mathcal{L} = 1.1887$) is marginal and costs memory and compute (block 512 forces batch down to 8 on longer sequences). We take 256.

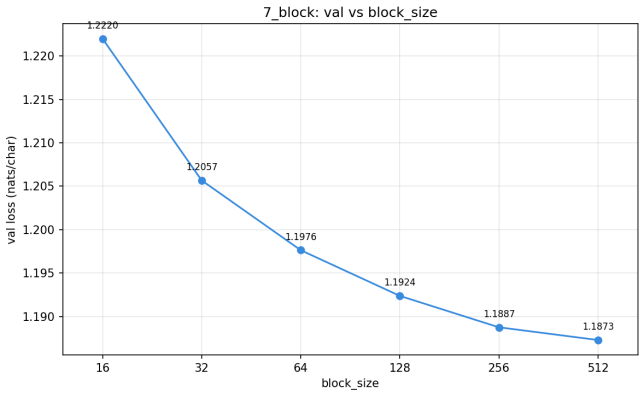

Figure 9: block sweep: validation loss vs block size (tps = 4096)

Choice. block = 256, batch = 16 (tps = 4096), $\mathcal{L} = 1.1887$.

2.7 Regularisation

We sweep weight decay against dropout (9_reg). Weight decay has **no measurable effect**: 0.0, 0.0.1, 0.1 give identical loss. This is expected under the Muon-hybrid, where Muon (the bulk of the weights) carries no weight decay and only the small AdamW-aux group (embeddings, head, norms) sees it. Dropout, on the other hand, clearly helps: 0.0 gives 1.2385, 0.1 gives 1.1886, and **0.2 is best at** $\mathcal{L} = 1.1871$.

Choice. dropout = 0.2 (weight decay left at its default, inert).

2.8 Phase A Optimal

Phase A takes the baseline from $\mathcal{L} = 1.7803$ to $\mathcal{L} = 1.1871$. The locked configuration, and what changed from baseline:

Component	Baseline	Phase A optimal
Optimizer	AdamW	Muon-hybrid
LR schedule	cosine	WSD (sqrt, decay 0.1, warmup 0.05)
Learning rate	6e-3	1e-2
Norm	LayerNorm	RMSNorm
Positional	learned	RoPE
Tokenizer	BPE	Unigram
Vocab size	16k	16k
Layers	6	12
`d_model`	512	384
Heads	8	6
Title masking	off	on
Block size	128	256
Batch size	64	16
Dropout	0.1	0.2

3 Phase B

With Phase A locked, we freeze its result as a second model variant, `gpt_v2`. `gpt.py` carried `if/else` for architectural ablations, `gpt_v2` now hardwires the winners (RMSNorm, RoPE, per-title masking, GELU MLP, biases kept) dropping the branching complexity. `GPTConfig` is shared between the two (`model/config.py`), training knobs still come from config, and `gpt_v2=True` selects the variant at train and load time. This keeps Phase B clean: new ideas are ablated against a fixed backbone.

We now experiment with a range of more interesting ideas, guided by the current shorting comings of our HN LLM model.

3.1 Alternative Attention Design

HN LLM is severely capacity-limited rather than memory-limited, hence the best attention mechanisms for it are those that inject better inductive biases (like relative positions) or expand the expressive capacity per token.

This rules out the usual efficiency variants (GQA, MQA, MLA): they shrink the memory we do not need, and cost quality. We instead swap the whole attention component for five candidates, all **iso-parameter**, ablated on the `gpt_v2` backbone at the Phase A optimal config (11_attn). A win therefore means a *better use of a parameter*, not more parameters.

Baseline (MHA) [18]. The locked `gpt_v2` attention: RoPE on q/k, per-title masked softmax. The reference all variants are measured against.

Single head. One head over the full `d_model` (`n_head=1`), parameter-identical to MHA. A lower bound that tests whether multi-head routing earns its keep: multiple heads give the model several attention subspaces, and this collapses them to one.

Value-residual [19]. Each layer mixes its values with the first layer's:

```
if v0 is not None:                    # v0 = layer-0 values,
    threaded down
    g = torch.sigmoid(self.lam)       # learned scalar per
     layer
    v = (1 - g) * v + g * v0
```

This gives a shortcut for the original (first-layer) value information to bypass intermediate layers, which mitigates the attention-concentration that deep stacks suffer and lets later layers re-access early features.

Output-gated [20]. An input-dependent sigmoid gate on the attention output, per head:

```
g = torch.sigmoid(self.gate(x))      # gate: Linear(d_model, n_head)
y = y * g[..., None]                 # modulate each head's contribution
```

The model learns, per token, how much each head's attention should contribute, adding expressivity (and reported training stability) for a tiny parameter cost.

Differential [21]. Two attention maps per head, subtracted:

```
q1, q2 = q.chunk(2, -1); k1, k2 = k.chunk(2, -1)   # split head_dim, rope each
y = attn(q1, k1, v) - lam * attn(q2, k2, v)        # cancel common-mode noise
```

The subtraction cancels the noise floor that softmax attention spreads over irrelevant tokens, sharpening attention to the few that matter. This is the clearest "better inductive bias" lever, and is exactly what a capacity-limited model wants: more signal per attention operation, as opposed to extra parameters.

The two views below show *what each mechanism does*. The **attention maps** (layer 8, same packed block) show how each shapes the softmax: MHA spreads its mass within a title, single-head is sparse and peaky, and differential is visibly cleaner—its subtraction strips the low-level noise floor.

Meanwhile the variants that leave the map untouched instead expose **learned knobs**, which read as their priorities: output-gating drives *early*-layer heads down and *deep*-layer heads up (a depth-dependent head weighting); value-residual holds ∼half of the first layer's values at every depth; and differential's subtraction λ *grows with depth* (0.24→0.64), i.e. there is more common-mode noise to cancel deeper in the stack.

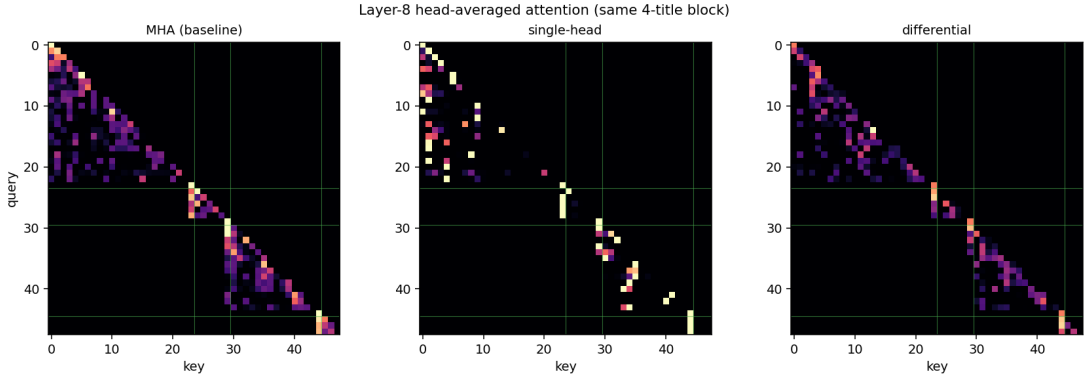

Figure 10: Layer-8 attention maps: MHA vs single-head vs differential

4

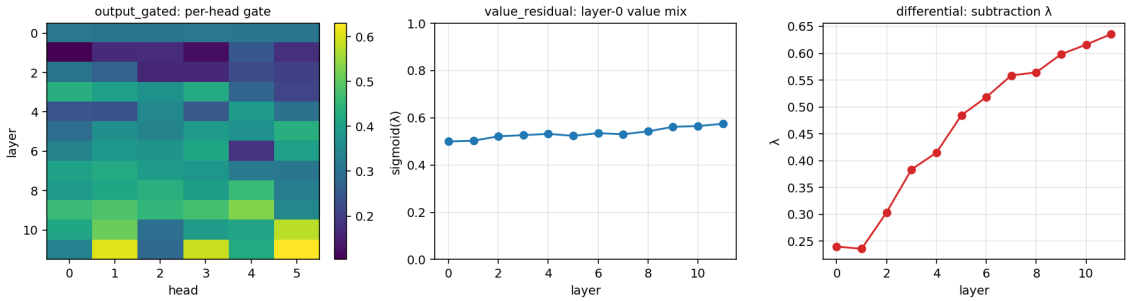

Figure 11: Learned knobs: output-gate per head, value-residual mix, differential λ

Results. Across the variants (10_attn) the per-head **output gate wins** (val 1.1814), ahead of value-residual (1.1847), plain MHA (1.1870), single-head (1.1898) and the differential transformer (1.1913). The pattern is clean: a *gentle, learnable, per-head modulation* of a working attention beats both the unmodified mechanism and the more aggressive restructurings. We carry output-gating forward.

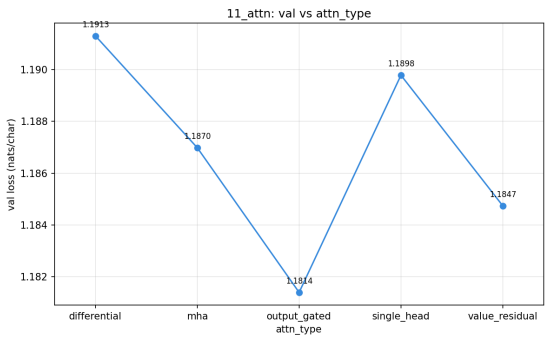

Figure 12: Attention variants: validation loss

3.2 Cross-layer residual paths

Beyond the single residual within each block, we test three ways to add *extra* information paths across layers, all gated and iso-parameter, ablated on the output-gated backbone (11_residual).

U-Net skips [22]. Symmetric encoder-decoder skips: the second half of the stack reads a gated copy of the mirrored first-half block's activations.

```
if i >= h: # decoder half
    x = x + skip_gates[i-h] * saved[h-1-(i-h)]    # mirror
     of the encoder block
```

Borrowed from U-Net in vision, where encoder-decoder skips preserve fine detail. Here it is an information (and gradient) highway: deep layers can re-access early representations that would otherwise have to survive every intermediate block. Gates init 0 (identity at start), so the path only turns on if it helps.

Embedding shortcut [23]. Feed the raw token embeddings straight into the deep layers:

```
if i >= h:
    x = x + emb_gates[i-h] * e0 # e0 = token_emb(idx)
```

This gives the later layers a direct path to token identity that bypasses the query-key routing, so token-level information never has to be reconstructed from deep activations.

LayerScale [24]. A learned per-channel scale on each residual branch (init 0.1):

```
x = x + ls1 * attn(...)
x = x + ls2 * mlp(...)
```

Lets the network choose, per channel, how much each layer contributes, which eases optimisation of the deeper stack.

We expect these to be modest here and ablate them honestly: value-residual (a related value-stream shortcut) already lost to output-gating, which is mild evidence that "more shortcut" was not the binding constraint. U-Net and the embedding shortcut are different paths (residual-stream and embedding), so they are still worth a clean on/off.

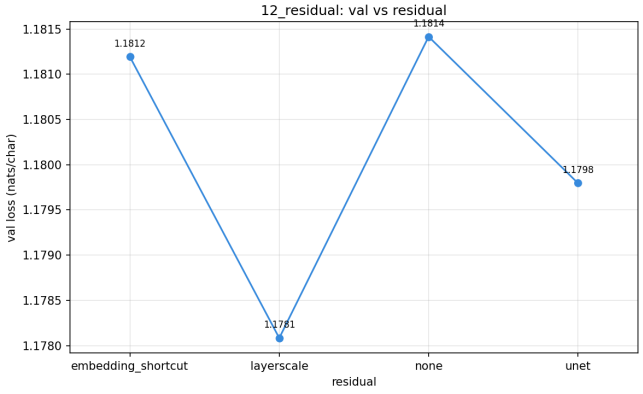

Figure 13: Cross-layer residual paths: validation loss

Decision. On validation, **LayerScale wins** narrowly (1.1781) over U-Net (1.1798), the embedding shortcut (1.1812) and the plain backbone (1.1814). As predicted the cross-layer paths help little, and only LayerScale's per-channel residual gating has a clear edge.

3.3 Per-head attention temperature

Motivation. This idea is entirely informed by *our own* Phase B results. Lining up the attention and cross-layer ablations, a pattern emerges: the changes that helped (output-gating, value-residual) are all *small, per-head, learnable dials on a working attention*, while the ones that did not (the differential mechanism, single-head, and the cross-layer residual paths) either replace the attention mechanism or bolt an extra pathway on the side. Every dial we tried, though, acts on the **output** side (gate the output, blend the values). The one axis we never gave the model a dial on is the **attention distribution itself**.

Idea. A per-(layer, head) learned scalar `g`, applied as a temperature on the scores:

$$\text{score} = \frac{q \cdot k}{\sqrt{d}}\, e^{g_{l,h}}, \qquad g \text{ zero-initialised.}$$

Since scaling `q` scales the dot-product scores, $e^g > 1$ sharpens a head's softmax (peakier, more copy-like) and $e^g < 1$ softens it (flatter, more averaging). At `g=0` the factor is 1, so the model is bit-identical to the output-gated baseline at step 0 and the gate only moves if the optimiser finds it useful.

```
# per head, zero-init
self.log_temp = nn.Parameter(torch.zeros(n_head))

# after RoPE, a per-head scalar commutes with the rotation
q = q * torch.exp(self.log_temp).view(1, -1, 1, 1)
```

Why it suits this dataset. Titles are ∼13 tokens and title-masked, so every head reasons over a tiny, independent context. In that regime there is a genuine tension between heads that want to *sharply copy* the previous token (short-range induction) and heads that want to *summarise* the whole title, and a single fixed softmax temperature forces a compromise. A per-head temperature lets each head pick its own sharpness, which is exactly the kind of per-head dial our own results say the model rewards, on the one attention axis we had not probed. It costs $n_{\text{layer}} \times n_{\text{head}}$ scalars (72), and the learned `g` per head is a readable diagnostic of which heads sharpen and which soften. (12_attn_temp)

Results. No noticeable impact: the learned per-head temperature leaves validation loss essentially unchanged (within run-to-run noise), so we do not adopt it.

attn temperature	val loss
off	1.1781
on (per-head, learned)	1.1776

3.4 The train–val gap (overfitting)

A sweep of the AdamW-aux learning rate (the embeddings/tied-head/norms group, run at `lr_hybird` against the matrices' 1e-2) was meant to tune that knob, but instead exposed the binding constraint:

`lr_hybird`	train	val	gap
3e-4	1.048	**1.182**	0.134
1e-3	0.898	1.201	0.303
3e-3	0.838	1.234	0.396

Raising the aux LR drives **train loss down (1.05 → 0.84) while val rises (1.18 → 1.23)**, the gap nearly triples. The model is **generalisation-bound, not optimisation-bound**: it can already drive train loss well below val, so the remaining lever is regularisation (closing the gap), not more optimiser power (lowering the floor). The effect is largest when we push the *head*, which points the next step at regularising the output distribution directly.

3.5 R-Drop and the capacity frontier

The gap analysis says we need regularisation that lowers the *floor*, not more optimiser power. Plain regularisers fail that test: dropout and weight decay all close the gap by *raising* train loss while val stays flat or worsens, they trade train fit for nothing. **R-Drop** [25] is conceptually different. Each batch goes through the model twice with independent dropout masks, and a consistency penalty, the symmetric KL between the two output distributions, is added to the usual cross-entropy:

$$\mathcal{L} = \tfrac{1}{2}(\mathrm{CE}_1 + \mathrm{CE}_2) + \lambda \cdot \tfrac{1}{2}\big(\mathrm{KL}(p_1 \| p_2) + \mathrm{KL}(p_2 \| p_1)\big)$$

```
# two passes, different dropout masks
lg1, l1 = model(x, y)
lg2, l2 = model(x, y)
kl = sym_kl(softmax(lg1), softmax(lg2))
loss = 0.5 * (l1 + l2) + rdrop * kl
```

Forcing the two dropout sub-models to agree is an implicit ensemble: it regularises the *function* the network computes rather than just injecting activation noise, which is why it lowers the floor where the others only moved the gap.

`r-drop` λ	val
0	1.1780
2	1.1543
4	1.1608

On its own it takes val from 1.1776 to **1.1543** at d384 (weight $\lambda 2$), the only single change that improved the scored metric, and it does so by improving generalisation.

R-Drop reopens capacity. d384 won the original shape search only because larger models overfit; once R-Drop controls that, capacity has the potential to pay off again. The 3×3 grid plot below over width × R-Drop weight (13_rdrop_size) shows it cleanly: with R-Drop *off*, width does nothing (d384/d512/d640 all sit at 1.175–1.181, d640 the worst); with R-Drop *on*, the larger models win, the best being **d640 + R-Drop = 1.1448**. So the win is a frontier, not a single setting, R-Drop turns the fixed d384 optimum into a capacity × regularisation trade, peaking at d640 (bf16 keeps it inside the 12 GB budget, matching fp32). Net **1.1776 → 1.1448** (−0.033), at the cost of one extra forward pass per step.

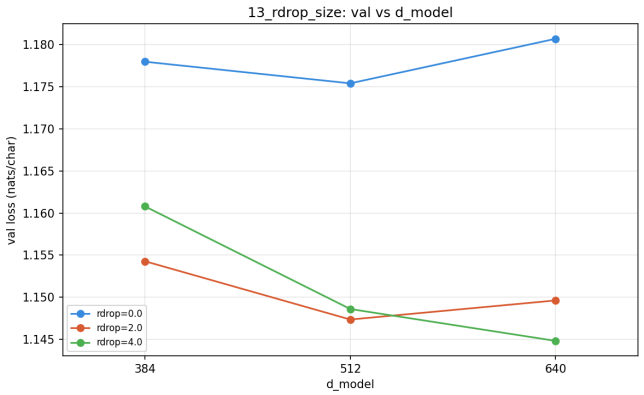

Figure 14: R-Drop x size sweep: validation loss vs d_model, one curve per R-Drop weight

Choosing the operating point. The lowest validation loss belongs to the largest model: d640 + R-Drop reaches **1.1448**. But size is not free, R-Drop already doubles the per-step forward and width adds on top, so we read val against *measured* wall-clock (plot below) rather than taking the best loss blindly. d640 pays 856 s for that 1.1448, whereas **d512 + R-Drop sits at the knee, 1.1473 for 599 s**, roughly 90% of the gain for 70% of the compute (R-Drop weight 2 vs 4 costs the same, since both run two passes). So while the largest model is the lowest-loss point, the *compute-optimal* one is **d512 + R-Drop (2× passes)**, and that is the configuration we take.

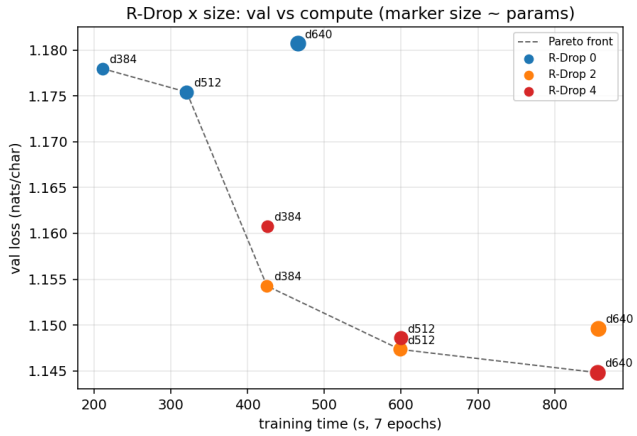

Figure 15: R-Drop x size: validation loss vs training time (marker size ∼ params)

3.6 Challenging coordinate descent

Coordinate descent locked each Phase A choice early and in isolation—before the capacity unlock via R-Drop and

the other changes such as output-gating etc. Therefore, a choice rejected back then may pay off now, so as a final check we go back and revisit the changes we left behind: from the full d512 best we flip one Phase-A-rejected choice back on at a time (the three Llama-style flags SwiGLU, QK-norm, bias-off) and ask whether it now lowers validation loss. If a rollback beats the reference, coordinate descent missed an interaction; if none does, the locks were robust (14_rollback).

rollback (one change vs reference)	val
best (d512 reference)	1.1479
SwiGLU	1.1506
QK-norm	1.1495
bias-off	**1.1464**

Of the three, only **bias-off** lowers validation loss (1.1464 vs 1.1479, −0.0015); SwiGLU and QK-norm both regress on the scored metric. The gain is small and inside the run-to-run band, but bias-off does not just edge the number: dropping the biases from every linear layer and norm makes the block strictly simpler, and it is the well-grounded Llama choice we had only rejected earlier because it was flat at depth 6. So we adopt **bias-off**: it lowers validation loss while making the block simpler and more principled, and we carry it forward as the configuration the next stage builds on.

3.7 Hybrid Muon Weight Decay

Revisiting left-behind choices in the same spirit, the most promising one is the regulariser our reg sweep (9_reg) had found completely inert: weight decay. It was inert for a structural reason. Under Muon-hybrid, decoupled weight decay only ever reached the small AdamW-aux group (embeddings, tied head, norms, biases); the 2D weight matrices that **Muon** optimises carried *no weight decay at all* and grew unconstrained across the 7 epochs.

To combat this, we decouple a small weight decay onto the Muon group directly, inside the optimiser: after each orthogonalised step, $p \leftarrow (1 - \eta\lambda)\, p$ scaled by the live learning rate η. This is the canonical "scalable Muon" fix [26], and recent theory shows Muon-with-weight-decay implicitly optimises under a *spectral-norm* constraint on the weight matrices [27], so it is geometry-matched to what Muon already does to the update, rather than the magnitude-only penalty AdamW applies. We sweep λ on the final best model (15_muon_wd); $\lambda = 0$ is the no-decay baseline (1.1462 here), so the curve both shows the gain and confirms we pick the optimum rather than assume it.

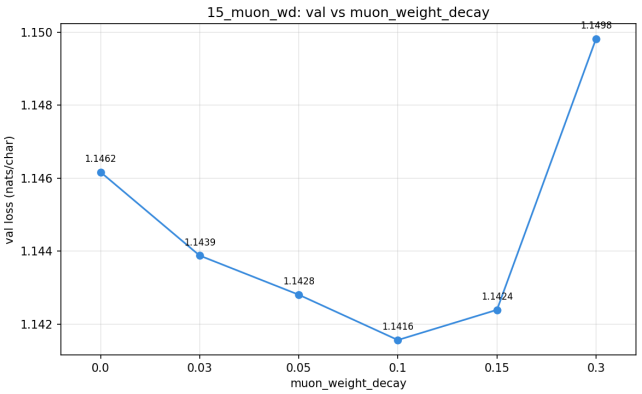

Figure 16: Validation loss vs Muon weight decay (15_muon_wd)

The sweep is a clean U. With no decay ($\lambda = 0$) we sit at 1.1462; decay lowers validation loss monotonically to a minimum of $\mathcal{L} = 1.1416$ at $\lambda = 0.1$ ($0.03 \to 1.1439$, $0.05 \to 1.1428$, $0.15 \to 1.1424$), then over-regularises at $\lambda = 0.3$ (1.1498). The −0.0046 gain is the largest single move of Phase B after R-Drop itself, and it closes the gap from the *right* side: train rises ($0.92 \to 0.94$) while val falls, the signature of genuine regularisation rather than the train-fit-for-nothing trade the plain regularisers gave. The critic stays inside its band ($1.08 \to 1.12$), so generation is not sacrificed for the loss. The lesson is simple: weight decay was never useless here, it was just being applied everywhere *except* where the parameters actually were.

3.8 Phase B Optimal

Phase B takes the Phase A optimal from $\mathcal{L} = 1.1871$ to $\mathcal{L} = 1.1416$ (a −0.0455 improvement), and every gain comes from a *better use of a fixed parameter budget* rather than more parameters. Four changes stuck. We replaced attention with a per-head **output gate**, added **LayerScale** on the residual branches, and re-opened capacity with **R-Drop**: once R-Drop controls the overfitting that capped Phase A, width pays off again, so the d384 shape moves back up to d512 (8 heads) read at the compute knee rather than the lowest-loss point. Finally, decoupling a small **weight decay onto the Muon matrix group** ($\lambda = 0.1$, Section 3.7) lands the largest late gain, regularising exactly the 2D weights Muon had left unconstrained. We also carry the **bias-free** block from the rollback check. Ideas that did not earn their place, the per-head attention temperature and the U-Net and embedding-shortcut residual paths, are dropped.

Component	Phase A optimal	Phase B optimal
Attention	MHA	Output-gated
Residual path	none	LayerScale
R-Drop	off	on (weight 2)
Muon weight decay	0	0.1
`d_model`	384	512
Heads	6	8
Biases	kept	off

Everything else carries over unchanged from Phase A.

4 Performance Analysis

We step back and analyse the final model through the three lenses from Appendix B: the scored validation loss, the learned critic (does low loss actually mean good generation?), and the train–val gap (is the model still overmemorising?).

Validation and critic track together. At each stage we selected predominatly on validation loss. We now plot what the autoreggresive scores (critic judge) yield, to ensure generation has also improved.

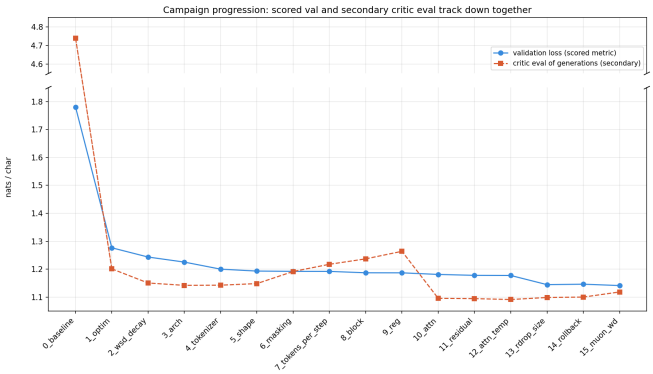

Figure 17: Validation loss and critic loss across the 16 campaign stages

The two fall in sync: from the baseline both collapse into the ∼1.1–1.3 band and then track within ∼0.1 of each other the rest of the way, with the critic reaching its best in Phase B (10–15), exactly where the generation-focused ideas, output-gating and R-Drop land. The one place they part is the mid-campaign data-handling stages (6–9), where small validation gains did not improve generation. The final stage (15, Muon weight decay) takes validation loss to its campaign low of 1.1416 while the critic ticks up only slightly (to 1.12) and stays well inside its band: the floor-lowering regulariser buys loss without costing generation.

Importantly, there was never any large disagreement between the critic and next token cross-entropy loss, meaning the search stayed on a genuinely good path.

The overfitting story. Plotting the selected model's final train and validation loss at every stage allows us to observe generalisation behaviour.

Figure 18: Per-stage train vs validation loss

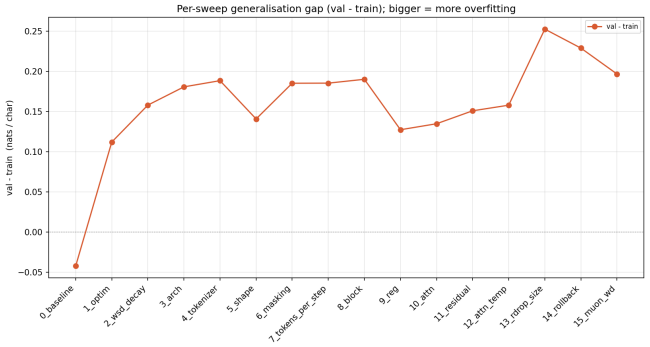

Figure 19: Per-stage generalisation gap, val - train

Read together, these say the gap is not a fault to be minimised but a consequence of *using* capacity. The baseline starts undertrained (train just above val, a slightly negative gap); as the model improves, train falls faster than val and the gap widens to a ∼0.18–0.19 plateau through the middle stages, the inherent memorisation of ∼6.4M tokens over 7 fixed epochs. It then spikes to ∼0.25 exactly where R-Drop unlocks capacity (13–14): the model finally has the room to fit train hard (train ∼0.89) while val keeps dropping. The final Muon-weight-decay stage then pulls the gap back from the *right* side (0.25 → 0.20, train rising while val falls), the signature of regularisation that lowers the floor rather than trading train fit for nothing. The through-line of the whole campaign is that the levers which moved validation loss were the ones that controlled *how* capacity is used (R-Drop, Muon weight decay), not the ones that simply removed it (smaller models, parameter sharing, see Appendix C).

Final model. The best model reaches validation loss **1.1416**. A handful of samples conditioned on *Show HN* & *Ask HN*:

Best model (val = 1.1416)
Show HN: destructive bitmaps for Apple II (UK)
Show HN: django-inspired UI utility with web libraries for Node.js
Ask HN: How to find your first developer?
Ask HN: How to use a ROI module for your team?

The titles are well-formed and recognisably Hacker-News-like (Ask HN / Show HN framings, product launches, opinion pieces), if not always factual expected for a 46M-parameter model trained on this much data.

We summarise the set of attempted optimisations which yielded no performance in Appendix C for brevity.

5 Rust-based Inference Engine

As an interesting side-project I set out to try build a Rust based inference engine for our best performing model. This was purely for learning and experimenting (inspired by a recent Maincode post on writing their own from-scratch Rust inference stack). The codebase draws from ideas in llama2.rs; it is a small, dependency-light CPU engine that reproduces our model's forward pass and

generates titles. It lives in `inference/` and is deliberately kept out of the graded submission.

How it is wired up

The trickiest practical bit is just getting the weights and tokenizer out of Python and into Rust cleanly. Our checkpoint (`model.pt`) is a PyTorch pickle, which is painful to read from Rust, so a tiny export script (`export.py`) converts a finished run into a `safetensors` file (a trivial header-plus-raw-floats format) alongside a `meta.json` that records the model shape. The tokenizer is easier: HuggingFace's `tokenizers` library is itself written in Rust, so we load the exact `tokenizer.json` saved at train time and get byte-for-byte encode/decode parity for free, with no retraining or re-implementation.

From there the engine is a handful of small files:

- `constants.rs` bakes the model shape (dims, layers, heads) in at compile time, and sanity-checks them against `meta.json` at load so we cannot run a binary built for the wrong model.
- `math.rs` holds the numeric kernels: matvec, RMSNorm, exact-erf GELU, sigmoid, softmax, and the RoPE rotation. Plain `f32`, written for clarity first.
- `model.rs` loads the `safetensors` weights and implements the forward pass, reproducing our locked `gpt_v2` block exactly: RMSNorm, output-gated attention, layerscale residuals, the GELU MLP, RoPE, and the tied embedding head.
- `cache.rs` is the KV cache, so each new token attends over the stored keys/values rather than recomputing the whole context.
- `sample.rs` turns logits into the next token (temperature, top-k, greedy) with a small seeded PRNG for reproducibility, and `main.rs` is the CLI that ties it together.

Decoding is one token at a time through the KV cache. Getting the details right matters more than the speed here: the RoPE layout, the biases, the per-head output gate, and the layerscale factors all have to match the trained model or the samples quietly drift. With those in place the engine generates perfectly coherent HN-style titles (e.g. *"Show HN: Jekyll, a Python script to make money from your browser"*), which is a good sign the architecture was reproduced faithfully.

A quick speed comparison

For fun we compared our engine against the reference PyTorch model, both decoding 128 greedy tokens on CPU (8 threads). Parallelising the matvec across cores with `rayon` is the one optimisation we bothered with, and it is enough to come out ahead:

Engine	tok/s
Rust (multi-core)	121
PyTorch (CPU)	56

The headline is roughly 2.2×, but this is not an apples-to-apples comparison and we should not read too much into it. Our engine uses a KV cache while PyTorch's `generate` recomputes the full context at every step.

Closing thoughts

This was a fun way to understand the model from the bottom up, but it is a toy: a lot more work would be needed to make it genuinely useful. The obvious next steps are SIMD on the dot products, an int8 weight path (decode is memory-bound, so fewer bytes per token is the real lever), and eventually actual GPU integration. That said, on a model this small none of it would be especially impactful, which is rather the point: the interesting engineering only starts to pay off once the model is large enough to need it.

References

[1] S. J. Wright, "Coordinate descent algorithms," *arXiv preprint arXiv:1502.04759*, 2015.

[2] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," *arXiv preprint arXiv:1711.05101*, 2019.

[3] K. Jordan *et al.*, *Muon: An optimizer for the hidden layers of neural networks*, https://kellerjordan.github.io/posts/muon/, 2024.

[4] S. Hu *et al.*, "MiniCPM: Unveiling the potential of small language models," *arXiv preprint arXiv:2404.06395*, 2024.

[5] H. Touvron *et al.*, "LLaMA: Open and efficient foundation language models," *arXiv preprint arXiv:2302.13971*, 2023.

[6] B. Zhang and R. Sennrich, "Root mean square layer normalization," *arXiv preprint arXiv:1910.07467*, 2019.

[7] J. Su *et al.*, "RoFormer: Enhanced transformer with rotary position embedding," *arXiv preprint arXiv:2104.09864*, 2021.

[8] N. Shazeer, "GLU variants improve transformer," *arXiv preprint arXiv:2002.05202*, 2020.

[9] A. Henry *et al.*, "Query-key normalization for transformers," *arXiv preprint arXiv:2010.04245*, 2020.

[10] J. Hoffmann *et al.*, "Training compute-optimal large language models," *arXiv preprint arXiv:2203.15556*, 2022.

[11] J. Kaplan *et al.*, "Scaling laws for neural language models," *arXiv preprint arXiv:2001.08361*, 2020.

[12] R. Sennrich, B. Haddow, and A. Birch, "Neural machine translation of rare words with subword units," *arXiv preprint arXiv:1508.07909*, 2016.

[13] M. Schuster and K. Nakajima, "Japanese and korean voice search," in *ICASSP*, 2012.

[14] T. Kudo, "Subword regularization: Improving neural network translation models with multiple subword candidates," *arXiv preprint arXiv:1804.10959*, 2018.

[15] A. Liu *et al.*, "SuperBPE: Space travel for language models," *arXiv preprint arXiv:2503.13423*, 2025.

[16] Y. Tay *et al.*, "Scale efficiently: Insights from pre-training and fine-tuning transformers," *arXiv preprint arXiv:2109.10686*, 2022.

[17] J. Dong *et al.*, "Flex attention: A programming model for generating optimized attention kernels," *arXiv preprint arXiv:2412.05496*, 2024.

[18] A. Vaswani *et al.*, "Attention is all you need," *arXiv preprint arXiv:1706.03762*, 2017.

[19] Z. Zhou *et al.*, "Value residual learning for alleviating attention concentration in transformers," *arXiv preprint arXiv:2410.17897*, 2024.

[20] Z. Qiu *et al.*, "Gated attention for large language models," *arXiv preprint arXiv:2505.06708*, 2025.

[21] T. Ye *et al.*, "Differential transformer," *arXiv preprint arXiv:2410.05258*, 2024.

[22] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional networks for biomedical image segmentation," *arXiv preprint arXiv:1505.04597*, 2015.

[23] K. Jordan *et al.*, *Modded-nanogpt: Nanogpt speedrun*, https://github.com/KellerJordan/modded-nanoGPT, 2024.

[24] H. Touvron *et al.*, "Going deeper with image transformers (CaiT)," *arXiv preprint arXiv:2103.17239*, 2021.

[25] X. Liang *et al.*, "R-Drop: Regularized dropout for neural networks," *arXiv preprint arXiv:2106.14448*, 2021.

[26] J. Liu *et al.*, "Muon is scalable for LLM training," *arXiv preprint arXiv:2502.16982*, 2025.

[27] T. Pethick *et al.*, "Muon optimizes under spectral norm constraints," *arXiv preprint arXiv:2506.15054*, 2025.

[28] M. Strathern, *Improving ratings: Audit in the British university system*, European Review, 1997.

[29] J. Salazar, D. Liang, T. Q. Nguyen, and K. Kirchhoff, "Masked language model scoring," *arXiv preprint arXiv:1910.14659*, 2019.

[30] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," *arXiv preprint arXiv:1810.04805*, 2018.

[31] F. Gloeckle *et al.*, "Better & faster large language models via multi-token prediction," *arXiv preprint arXiv:2404.19737*, 2024.

[32] DeepSeek-AI, "DeepSeek-V3 technical report," *arXiv preprint arXiv:2412.19437*, 2024.

[33] I. Provilkov, D. Emelianenko, and E. Voita, "BPE-Dropout: Simple and effective subword regularization," *arXiv preprint arXiv:1910.13267*, 2019.

[34] C. Szegedy *et al.*, "Rethinking the inception architecture for computer vision," *arXiv preprint arXiv:1512.00567*, 2016.

[35] P. Izmailov *et al.*, "Averaging weights leads to wider optima and better generalization," *arXiv preprint arXiv:1803.05407*, 2018.

[36] P. Foret *et al.*, "Sharpness-aware minimization for efficiently improving generalization," *arXiv preprint arXiv:2010.01412*, 2020.

[37] Z. Li *et al.*, "NorMuon: Making muon more efficient and scalable," *arXiv preprint arXiv:2510.05491*, 2025.

[38] L. Wan *et al.*, "Regularization of neural networks using DropConnect," in *ICML*, 2013.

[39] Z. Lan *et al.*, "ALBERT: A lite BERT for self-supervised learning of language representations," *arXiv preprint arXiv:1909.11942*, 2019.

[40] Y. Cui *et al.*, "Class-balanced loss based on effective number of samples," *arXiv preprint arXiv:1901.05555*, 2019.

A Engineering

The repository is organised around a single benchmark engine and a declarative experiment config.

```
mainrun/
├── train.py          # training loop + Hyperparameters (the entrypoint, ‘task train‘)
├── bench.py          # sweep engine: iso-param solver, FLOP/token accounting, resume + caching
├── experiments.py    # declarative Sweep configs, one per run in sweeps/ below
├── sample.py         # autoregressive generation from a saved run
├── critic.py         # MLM critic: train + score (secondary eval)
├── ...
└── model/            # the architecture and its moving parts
    ├── gpt.py        # GPT: attention (RoPE, QK-norm, title masking), RMSNorm, SwiGLU, ablation flags
    ├── muon.py       # Muon optimizer
    ├── optimizer.py  # optimizer + LR-schedule builder (Muon-hybrid, WSD)
    ├── tokenizer.py  # tokenizer training: bpe | unigram | wordpiece | superbpe
    └── load.py       # rebuild a saved run (model.pt + tokenizer.json) for sampling/eval
sweeps/               # one self-contained folder per run (config, weights, tokenizer, logs, plots)
├── 0_baseline/
├── 1_optim/          # optimizer sweep
├── 2_wsd_decay/      # LR-schedule sweep
├── 3_arch/           # modern-block ablation
├── 4_tokenizer/      # iso-param tokenizer sweep
└── ...
inference/            # rust based infernece engine (toy project)
tools/
└── superbpe/         # vendored Rust ‘tokenizers-superbpe‘ fork (enables two-stage SuperBPE)
```

The numbered `sweeps/` folders mirror the Phase A sections of this report. Each experiment is defined declaratively as a `Sweep`:

```
"tok": Sweep(
    axes={"token_type": ["bpe", "unigram"], "vocab_size":
     [16_000, 32_000]},  # cartesian grid
    hold={"optim_alg": "muonhybrid", "lr": 1e-2},
                        # fixed across the sweep
    resolve=solve_width(budget=32_000_000),   # derive
     d_model/n_head to hit a param budget
    x="vocab_size", group="token_type",       # how rows
     map onto the plot
)
```

Key properties:

- **Iso-parameter sweeps.** `solve_width` solves for `d_model`/`n_head` to hit a fixed parameter budget at each point, so tokenizer and shape comparisons are fair at equal capacity.
- **Iso-tokens/step.** Batch size is auto-derived from a fixed `tokens_per_step`, holding step count and per-step compute constant so a sweep isolates the variable under test (relaxed deliberately for the batch/block study).
- **Reproducible & resumable.** Fixed seed; every run is tagged with the commit short-hash and saves its config, weights, tokenizer, and logs; sweeps cache results and skip completed cells on resume.
- **Compute accounting.** Approximate train FLOPs and exact token counts are logged alongside each run, so design decisions can be read against the hardware budget, not just loss.

For each run, we save the `mainrun.log`, `tokenizer.json`, `model.pt` and `config.json`—this allows us to evaluate and inspect checkpoints via the `load.py` class.

We use a single GTX 4070 Super (12 GB) for experiments.

B Evaluation

When a measure becomes a target, it ceases to be a good measure (Goodhart [28]). Teacher-forced next-token validation loss is our *main* optimisation target, but on its own it never tests autoregressive output (whole titles). So we add two **secondary** metrics, built on autoregressive analysis. They are explicitly *not* optimisation targets – but **sanity checks**: do lower-loss models actually generate coherent, title-like text, and is the search on a generally good path?

1. **Validation loss.** Primary, the scored metric and the only thing we optimise: cross-entropy in nats/char on the held-out split, per-character so it stays comparable across tokenizers of different vocab sizes.
2. **Qualitative generation (`sample.py`)** *(secondary).* We generate titles autoregressively (primed with `<eos>`, the de-facto start token) and read them, checking that low loss corresponds to coherent, well-formed titles.
3. **Learned critic (`critic.py`)** *(secondary, Phase B).* Once generation is strong enough to score, we train a small BERT-style encoder from scratch on the real titles and freeze it into a fixed ruler, then score any generation by its pseudo-log-likelihood [29], lower nats/char meaning more title-like. It is reference-free and tokenizer-agnostic (it scores decoded *text*), so it stays a consistent ruler across models; its architecture and training are detailed below.

We save notable qualitative generations (2) in Appendix D, and run a full critic evaluation at the end of

Phase B (Section 4) that plots validation loss and critic loss together across the whole campaign.

B.1 The learned critic

Validation loss is teacher-forced, so it never watches the model run freely: a model can post a low loss yet still degenerate once it has to generate a whole title unprompted. The critic is built to catch exactly that. It is trained once on the real titles and then frozen into a fixed ruler that scores any generation by how *title-like* it reads, independently of the model that produced it.

Architecture. A pre-norm, *bidirectional* Transformer encoder (no causal mask, so every position attends over the whole title): 8 layers, $d_{\text{model}} = 384$, 6 heads (head-dim 64), a GELU feed-forward of width $4d$, learned absolute position embeddings, and tied input/output embeddings, roughly 17M parameters. It carries its *own* fixed 8k BPE tokenizer and a block size of 32 (titles are short; 32 covers ∼99% of them). Crucially it tokenises and scores decoded *text*, so it is invariant to whichever tokenizer the model-under-test used, which matters because we sweep tokenizers across the campaign.

Objective. Masked language modelling in the BERT style [30] with the 80/10/10 rule (of the chosen positions, 80% become `[mask]`, 10% a random token, 10% are left unchanged), scoring cross-entropy only on the masked slots. We raise the masking rate to **30%** from BERT's 15%: a title is only ∼13 tokens, so 15% would leave barely two prediction targets per example.

Training. From scratch for **25 epochs** on 100k titles taken from the *train* split only, so the validation titles stay unseen and can later serve as an unbiased "real" anchor for the critic scores. Optimisation is AdamW (learning rate 5×10^{-4}, weight decay 0.01, batch size 128) with a linear warmup over the first 5% of steps and a cosine decay to zero after.

Scoring. A generation is scored by its pseudo-log-likelihood [29]: mask each position in turn, read $-\log p(\text{token} \mid \text{rest})$ at that slot and sum, reported in nats/char so it is comparable across titles of different length (the n single-mask variants of a title are run together in one batched forward pass). Lower is more title-like. The critic never touches the scored model, `evaluate()`, or the validation loss: it is a purely secondary, never-optimised signal.

C Failures

We follow Maincode's principle of reporting real, non-exaggerated results, which includes what did *not* work. Phase A is a clean coordinate descent where every choice has a logged sweep; Phase B, by contrast, is a search over open-ended ideas, several of which simply had nothing good to show. We record them here rather than quietly dropping them, grouped by what each was reaching for: a lower loss *floor*, or a smaller generalisation *gap*. We do not detail each implementation, but cite the source idea.

C.1 Lowering the loss floor

These ideas tried to push the scored metric down by adding capacity, an extra objective, or a richer input on top of the locked backbone.

Multi-token prediction [31]: extra heads predict the token two and three positions ahead (title-aware). It regressed as depth grew, taking validation loss from 1.181 up to between 1.20 and 1.21.

Decoupled MTP modules [32]: a dedicated module per look-ahead feeding a shared head. Flat-to-worse, 1.181 → 1.187.

Value embeddings [23]: token-id embeddings injected directly into the attention values, giving the deep layers a direct path to token identity. No measurable effect (∼1.182, however many value tables we added).

Tokenizer regularisation [33]: stochastic re-segmentation of the input each epoch. Dropout helps BPE (1.205 → 1.191) but still loses to plain unigram (1.181).

Label smoothing [34]: soften the one-hot training target. Monotonically worse, 1.178 → 1.193 at $\varepsilon = 0.15$.

C.2 Closing the generalisation gap

Once R-Drop and the capacity frontier were in place the model was firmly generalisation-bound (train ∼0.92 vs val ∼1.15), so the search turned to anti-overfit ideas on the d512 + R-Drop backbone. Most washed against the noise floor (∼0.0003); only the decoupled weight decay of Section 3.7 actually moved the number. The instructive negatives:

Weight averaging (EMA / SWA) [35]: evaluate a running average of the weights. Washed, and in any case subsumed by the WSD anneal (best −0.0007).

Sharpness-aware minimisation (SAM / ASAM) [36]: ascend then descend each step to seek flat minima. Washed on top of R-Drop (≥ baseline).

NorMuon [37]: Muon with an added per-neuron normalisation that rescales each neuron's update by its running second moment (Adam-style) on top of the orthogonalised step. Washed: the bind was generalisation, not optimisation.

Spectral / orthogonality / nuclear penalties [27]: constrain the Muon weights' singular values through a loss term. Muon orthogonalises the *combined* gradient, so a penalty-in-loss is cancelled; only post-step operations (weight decay) bite.

DropConnect [38]: dropout applied to the weights rather than the activations, zeroing a random subset of connections on each pass. Hurts (1.147 → 1.156); the weight noise over-perturbs the two R-Drop passes, like input noise.

Cross-layer sharing (ALBERT) [39]: reuse one block across depth (2 to 12× fewer parameters). Worse, monotone with sharing (1.146 → 1.166).

Class-balanced loss [40]: up-weight rare-token cross-entropy. Worse, it trains a different objective than the plain-CE metric scores.

Untied embeddings: separate input and output embeddings (+8M parameters). Overfits hard, 1.146 → 1.216.

The common thread across both groups: on a small, data-limited model, ideas that **add capacity or a second objective** (MTP, value embeddings, untied embeddings, class-balanced loss) hurt, and ideas that simply **remove capacity** (parameter sharing, smaller models) trade val for nothing.

D Generation

Models we sampled using default settings in `sample.py`, unless specified otherwise.

D.1 Baseline

baseline
for a What. the, – : – and How your' (ing will to a 's a to and (to and ? Why Twitter

D.2 Optimizer

sgd + lr0.03
HN: Weber Facebook:L] The are Google of Is A How In to as the D the Can The The New a the A the G: How with Ask HN: I- to- for – What with the a the Is

adamw + lr0.0001
San Francisco's Facebook, they think need "16k" to it "WAT6 has been a new app" What is your app with a game-Firtations? Show HN: A Google's UK Ask HN: Is there you do you learn to use the world?

muon hybrid + lr0.01
The New Internet Ask HN: Need software/th-class computing? NASA's Privacy Analysis of Chinese Lab Is Going to Send Down Why We Need an Entrepreneu Show HN: Read what great engineering are you?